

Datos en movimiento

Cómo los nodos core de n8n reciben, transforman y guardan los datos de Mercado Austral, desde el formulario de pedidos hasta la base de datos.

APRENDIZAJE ESPERADO 3 · TEXTUAL DEL PLAN FORMATIVO

MANIPULAR DATOS UTILIZANDO NODOS FUNDAMENTALES DE N8N, PARA TRANSFORMAR INFORMACIÓN SEGÚN REQUERIMIENTOS ESPECÍFICOS DEL WORKFLOW.

PLAN FORMATIVO

PF1821 · Construcción de Agentes y Automatización con Herramientas Low Code

MÓDULO

MA04560 · FUNDAMENTOS DE N8N Y MANIPULACIÓN DE DATOS · 18 h

CASO DE ESTUDIO

Mercado Austral, empresa ficticia

TIEMPO DE LECTURA

20 minutos, más la autocomprobación

Qué vas a lograr

Al terminar esta lectura tendrás la base para demostrar los criterios de evaluación del aprendizaje esperado 3, textuales del plan formativo:

- 3.1** MANIPULA DATOS UTILIZANDO NODOS FUNDAMENTALES DE N8N, PARA TRANSFORMAR INFORMACIÓN SEGÚN REQUERIMIENTOS ESPECÍFICOS DEL WORKFLOW.
- 3.2** IMPLEMENTA TRANSFORMACIONES DE DATOS ENTRE FORMATOS JSON, CSV Y XML.
- 3.3** APLICA EXPRESIONES N8N PARA PROCESAR Y FILTRAR INFORMACIÓN.
- 3.4** CONECTA BASES DE DATOS BÁSICAS USANDO SUPABASE, DE ACUERDO CON LOS REQUERIMIENTOS DEL WORKFLOW.

Cómo está organizada

- | | |
|---|---|
| 1 Cómo viajan los datos | 6 Formatos |
| 2 Nodos core (1): modificar, filtrar y resumir | 7 Variables y configuración |
| 3 Nodos core (2): dividir, combinar y decidir | 8 Supabase en 5 minutos |
| 4 Expresiones | 9 CRUD con el nodo Supabase |
| 5 Tipos de datos | + En síntesis, práctica, autocomprobación y glosario |

Contenidos del plan y dónde se tratan

Contenido del plan formativo (textual)	Dónde
3. NODOS CORE DE N8N Y MANIPULACIÓN DE DATOS	Antes de empezar
SET: MODIFICACIÓN DE DATOS.	Sección 2
IF/SWITCH: LÓGICA CONDICIONAL.	Sección 3
MERGE: COMBINACIÓN DE DATOS.	Sección 3
FILTER: FILTRADO DE DATOS.	Sección 2
SUMMARIZE: AGREGACIÓN DE DATOS.	Sección 2
SPLIT: DIVISIÓN DE DATOS.	Sección 3
MANIPULACIÓN DE DATOS.	Secciones 1 y 5
EXPRESIONES N8N: SINTAXIS Y FUNCIONES.	Sección 4
TRANSFORMACIÓN DE FORMATOS.	Sección 6
MANEJO DE ARRAYS Y OBJETOS.	Sección 1
VARIABLES DE ENTORNO.	Sección 7

Contenido del plan formativo (textual)	Dónde
INTEGRACIÓN CON BASES DE DATOS.	Sección 8
SUPABASE: CONFIGURACIÓN BÁSICA.	Sección 8
OPERACIONES CRUD BÁSICAS.	Sección 9
MANEJO DE DATOS RELACIONALES	Sección 9

PARA COMENZAR

Antes de empezar

CONTENIDO DEL PLAN 3. NODOS CORE DE N8N Y MANIPULACIÓN DE DATOS

En el aprendizaje anterior armaste tu primer workflow: un trigger, un paso intermedio y una acción. Funcionaba, pero con datos tal como llegaban. En la práctica, los datos casi nunca llegan limpios: un correo viene en mayúsculas, una comuna trae un espacio de más, una cantidad llega como texto. Esta lectura trata de lo que pasa entre la entrada y la salida: cómo se **transforman** los datos para que el resultado sea confiable.

Vas a trabajar con el mismo caso de todo el módulo. **Mercado Austral** es una distribuidora ficticia de abarrotes que recibe pedidos por un formulario web. Hoy una persona los copia a una planilla y cada viernes suma a mano lo vendido por comuna, porque el sistema de facturación solo acepta archivos XML. Al terminar esta lectura sabrás qué nodo usar en cada parte de ese proceso y cómo escribir las expresiones que lo hacen funcionar.

EL PROCESO QUE VAS A AUTOMATIZAR



Lee con n8n abierto. Cada sección trae un ejemplo del caso que puedes reproducir en tu propia instancia, y los errores frecuentes que vas a encontrar en la actividad 1.

1 Cómo viajan los datos

CONTENIDO DEL PLAN MANIPULACIÓN DE DATOS. · MANEJO DE ARRAYS Y OBJETOS.

En n8n los datos viajan de un nodo a otro como una lista de **ítems**. Cada ítem es un objeto JSON: un conjunto de pares campo y valor, como `"comuna": "Ñuñoa"`. Si el formulario recibe un pedido, sale un ítem; si lees un archivo con ocho pedidos, salen ocho ítems. La mayoría de los nodos procesa cada ítem por separado y entrega la misma cantidad que recibió, salvo los que están hechos para cambiarla, como Filter o Summarize.

Dentro de un ítem, un campo puede contener un valor simple (texto, número, verdadero o falso), un **objeto** con sus propios campos o un **array**, que es una lista de valores. Un pedido con varios productos, por ejemplo, puede traer un array `productos` donde cada elemento es un objeto con nombre y cantidad. Manipular datos es, en buena parte, saber llegar a cada uno de esos niveles.

EJEMPLO · MERCADO AUSTRAL · Un pedido con dos productos

```
{
  "cliente": { "nombre": "Almacén El Sol", "email": "compras@elsol.test" },
  "comuna": "Temuco",
  "productos": [
    { "nombre": "Azúcar 25 kg", "cantidad": 10 },
    { "nombre": "Aceite 5 L", "cantidad": 6 }
  ]
}
```

`$json.cliente.email` devuelve el correo, `$json.productos[0].nombre` devuelve "Azúcar 25 kg" (los arrays se cuentan desde 0) y `$json.productos.length` devuelve 2.

Para ver los ítems usa los paneles de entrada y de salida de cada nodo. La vista **Table** sirve para revisar muchos ítems a la vez; la vista **JSON** muestra la estructura exacta, con comillas y anidación; la vista **Schema** muestra solo los nombres y tipos de los campos, y desde ella puedes arrastrar un campo a un parámetro.

2 Nodos core (1): modificar, filtrar y resumir

CONTENIDO DEL PLAN SET: MODIFICACIÓN DE DATOS. · FILTER: FILTRADO DE DATOS. · SUMMARIZE: AGREGACIÓN DE DATOS.

Edit Fields (Set) es el nodo que más vas a usar. Sirve para crear campos nuevos, cambiar el valor de los existentes o dejar solo los que necesitas. En el modo *Manual Mapping* defines cada campo con un nombre, un tipo (String, Number, Boolean, Array u Object) y un valor, que puede ser fijo o una expresión. La opción *Include Other Input Fields* decide si los campos que no tocaste siguen de largo o se descartan.

Filter deja pasar solo los ítems que cumplen una o más condiciones y descarta el resto. Es el nodo para limpiar la entrada: pedidos sin correo, cantidades en cero o registros de prueba. Un Filter no cambia los ítems, solo decide cuáles siguen, por lo que la salida puede tener menos ítems que la entrada.

Summarize agrupa y calcula. En *Fields to Split By* indicas por qué campo agrupar (por ejemplo, `comuna`) y en *Fields to Summarize* eliges qué calcular sobre cada grupo: *Sum*, *Count*, *Average*, *Min*, *Max*, entre otras. Recibe muchos ítems y entrega uno por grupo.

EJEMPLO · MERCADO AUSTRAL · El resumen del viernes

El archivo histórico trae 8 pedidos. El **Filter** con la condición "email no está vacío" descarta el pedido 4, que no tiene correo, y deja 7. Un **Edit Fields** calcula `total` como número y quita los espacios de `comuna`. El **Summarize**, agrupando por `comuna` con *Count* de `id` y *Sum* de `total`, entrega 4 ítems: Ñuñoa (2 pedidos, \$38.800), Maipú (2, \$208.600), Temuco (2, \$314.940) y Valparaíso (1, \$4.200). Suman \$566.540.

ERROR FRECUENTE · Agrupar antes de normalizar

Si el **Summarize** recibe "Ñuñoa" y "Ñuñoa " (con un espacio al final), los trata como dos comunas distintas. El resumen muestra dos filas para la misma comuna y ninguna está bien. Normaliza con **Edit Fields** **antes** de agrupar, nunca después.

3 Nodos core (2): dividir, combinar y decidir

CONTENIDO DEL PLAN SPLIT: DIVISIÓN DE DATOS. · MERGE: COMBINACIÓN DE DATOS. · IF/SWITCH: LÓGICA CONDICIONAL.

Split Out hace lo contrario de agrupar: toma un campo que contiene un array y lo convierte en ítems separados, uno por elemento. Es útil cuando un pedido trae una lista de productos y necesitas guardar cada producto como una fila. En *Fields To Split Out* indicas el campo, y con la opción *Include* decides si cada ítem nuevo conserva también los datos del pedido original.

Merge une dos flujos de datos que llegan por sus dos entradas. Tiene varios modos. *Append* pone los ítems de una entrada a continuación de los de la otra. *Combine* los junta en uno solo, y hay que elegir cómo: por un campo en común (*Matching Fields*), por posición (el primero con el primero) o con todas las combinaciones posibles. Para enriquecer datos, que es el uso más común, el modo correcto es combinar por un campo en común.

If y **Switch** envían cada ítem por un camino según una condición. **If** tiene dos salidas, verdadero y falso. **Switch** tiene tantas salidas como reglas defines. Aquí basta con saber para qué sirven; en el aprendizaje esperado 4 vas a trabajar con operadores, rutas y casos que no calzan con ninguna regla.

EJEMPLO · MERCADO AUSTRAL · Agregar la zona de despacho

Los pedidos llegan por la entrada 1 del **Merge** y la tabla `comunas` (con las columnas `comuna` y `zona`) por la entrada 2. Con *Combine* por *Matching Fields* sobre el campo `comuna` en las dos entradas, cada pedido sale con su `zona` ("RM" o "Regiones"). Después, un **If** con la condición "zona es igual a Regiones" separa los pedidos que van a despacho regional.

ERROR FRECUENTE · Pedidos clonados

Si el Merge queda en modo de todas las combinaciones, cada pedido se combina con **cada** comuna de la tabla: 6 pedidos y 40 comunas producen 240 ítems. La señal es clara si cuentas los ítems a la salida del nodo. La solución es combinar por el campo `comuna`.

4 Expresiones

CONTENIDO DEL PLAN EXPRESIONES N8N: SINTAXIS Y FUNCIONES.

Una **expresión** es un pedazo de JavaScript que n8n evalúa para cada ítem. Se escribe entre dobles llaves, `{{ }}`, en cualquier parámetro que esté en modo *Expression* (no *Fixed*). El editor de expresiones muestra una vista previa del resultado con los datos del ítem actual, así que puedes comprobar que funciona antes de ejecutar el nodo.

Las referencias que más vas a usar son `$json`, con los datos del ítem que entra al nodo; `${'Nombre del nodo'}.item.json`, con los datos del ítem equivalente en un nodo anterior; y `$now`, con la fecha y hora de la ejecución. Sobre esos valores aplicas funciones de texto y número como en JavaScript. n8n agrega además funciones propias que aparecen en el autocompletado al escribir un punto después de un valor.

Expresión	Qué devuelve
<code>{{ \$json.email.trim().toLowerCase() }}</code>	El correo sin espacios en los extremos y en minúsculas
<code>{{ \$json.comuna.trim() }}</code>	La comuna sin espacios al inicio ni al final
<code>{{ Number(\$json.cantidad) }}</code>	La cantidad convertida de texto a número
<code>{{ (\$json.total * 1.19).toFixed(0) }}</code>	El total con IVA, sin decimales (como texto)
<code>{{ \$now.toFormat('yyyy-MM-dd') }}</code>	La fecha de la ejecución, por ejemplo 2026-09-25
<code>{{ \$('Edit Fields').item.json.email }}</code>	El correo ya normalizado en el nodo Edit Fields

EJEMPLO · MERCADO AUSTRAL · El total del pedido

En el Edit Fields del workflow "Pedido a registro", el campo `total`, de tipo Number, vale `{{ Number($json.cantidad) * Number($json.precio_unitario) }}`. Para un pedido de 2 unidades de café a \$12.500, la vista previa muestra 25000 antes de ejecutar. Si muestra un texto o un error, la expresión o los tipos están mal.

ERROR FRECUENTE · La expresión que no se evalúa

Si en la salida ves literalmente `{{ $json.email }}` en vez del correo, el campo quedó en modo *Fixed*. Cambia el modo a *Expression*: el campo se pinta de otro color y aparece la vista previa.

5 Tipos de datos

CONTENIDO DEL PLAN MANIPULACIÓN DE DATOS.

Cada valor tiene un tipo: **texto** (String), **número** (Number), **booleano** (verdadero o falso), **fecha** y las estructuras que ya viste, objeto y array. El tipo importa más de lo que parece. En la vista JSON, un texto aparece entre comillas ("12500") y un número sin ellas (12500). Para una persona son lo mismo; para un nodo, no.

Los formularios y los archivos CSV entregan casi todo como texto. Si sumas dos textos en JavaScript, se concatenan: "3" más "2" da "32". Si comparas un texto con un número, la condición puede no cumplirse aunque el valor parezca correcto. Y una base de datos que espera un número entero rechaza la fila completa. Por eso conviene **declarar el tipo** en Edit Fields apenas entran los datos: `cantidad` y `precio_unitario` como Number, `email` y `comuna` como String.

Las fechas se guardan como texto en formato ISO 8601, por ejemplo `2026-09-25T18:00:00`. Así se ordenan bien y cualquier base de datos las entiende. `$now` entrega la fecha actual, que puedes formatear como necesites.

EJEMPLO · MERCADO AUSTRAL · El mismo pedido, antes y después

Antes de Edit Fields: `"cantidad": "2", "precio_unitario": "12500"`. Después, con los dos campos declarados como Number: `"cantidad": 2, "precio_unitario": 12500` y el campo nuevo `"total": 25000`, todos sin comillas. Esa es la señal de que los tipos quedaron bien.

ERROR FRECUENTE · El total que concatena

Si `total` quedó como String, el Summarize no suma: concatena los textos o marca un error de tipo. Revisa en la vista JSON si el valor tiene comillas y corrige el tipo en el Edit Fields que lo crea.

6 Formatos

CONTENIDO DEL PLAN TRANSFORMACIÓN DE FORMATOS.

n8n trabaja internamente en **JSON**, pero las empresas intercambian datos en muchos formatos. Transformar formatos es convertir los datos a JSON para procesarlos y, al final, convertirlos al formato que pide el destino.

Formato	Para leerlo	Para generarlo
JSON	Es el formato nativo: llega listo desde un webhook o una API	Sale tal cual en la respuesta de un webhook o de un HTTP Request
CSV	<i>Extract from File</i> , operación <i>Extract From CSV</i>	<i>Convert to File</i> , operación <i>Convert to CSV</i>
XML	Nodo XML , modo <i>XML to JSON</i>	Nodo XML , modo <i>JSON to XML</i>

Un archivo que llega por HTTP Request o por correo viaja como **dato binario** (el archivo tal cual), no como ítems. Por eso, para trabajar con un CSV descargado, primero lo lees con *Extract from File*: ese nodo convierte cada fila en un ítem y usa la primera fila como nombres de campo. Al revés, *Convert to File* toma los ítems y arma un archivo que puedes adjuntar a un correo o subir a una carpeta.

EJEMPLO · MERCADO AUSTRAL · De CSV a XML para facturación

El workflow "Resumen semanal" descarga `pedidos_historicos.csv` con un **HTTP Request** (respuesta en formato archivo), lo convierte en 8 ítems con **Extract from File**, filtra, normaliza y resume por comuna. Al final, el nodo **XML** en modo *JSON to XML* transforma los 4 ítems del resumen en el texto XML que acepta el sistema de facturación.

7 Variables y configuración

CONTENIDO DEL PLAN VARIABLES DE ENTORNO.

Hay valores que no deberían escribirse a mano dentro de un nodo: la URL de un servicio, el nombre de una tabla o, sobre todo, una clave. Si están repetidos en varios nodos, cambiarlos obliga a buscarlos uno por uno; si son secretos, quedan expuestos en el workflow exportado y en cualquier captura de pantalla.

n8n resuelve esto de dos formas. Las **credenciales** guardan las claves de acceso a otros servicios de manera cifrada, fuera del workflow: el nodo solo sabe qué credencial usar, nunca ve la clave. Las **variables de entorno** son valores definidos en el servidor donde corre n8n; en una instancia propia, si el administrador lo habilita, se leen con `$env.NOMBRE_VARIABLE`. Algunos planes de n8n ofrecen además *Variables* compartidas, que se leen con `$vars.NOMBRE`.

EJEMPLO · MERCADO AUSTRAL · Un valor que cambia entre prueba y producción

Mercado Austral prueba sus workflows contra una tabla `pedidos_prueba` y en producción usa `pedidos`. En vez de escribir el nombre en el nodo, la instancia define la variable `TABLA_PEDIDOS` y el nodo usa `{{ $env.TABLA_PEDIDOS }}`. El mismo workflow sirve en los dos ambientes sin editarlo.

IDEA CLAVE

Una regla simple: si un valor es secreto, va en una credencial; si cambia según el ambiente, va en una variable; si es parte de la lógica, va en el workflow.

8 Supabase en 5 minutos

CONTENIDO DEL PLAN INTEGRACIÓN CON BASES DE DATOS. · SUPABASE: CONFIGURACIÓN BÁSICA.

Una planilla sirve para mirar datos; una **base de datos** sirve para que varios procesos los usen sin pisarse. Integrar n8n con una base de datos permite guardar cada pedido apenas llega, consultarlo después y relacionarlo con otros datos. **Supabase** es un servicio que entrega una base de datos PostgreSQL en la nube, con un editor de tablas en el navegador, y n8n tiene un nodo propio para usarla.

La configuración básica tiene tres pasos. Primero, crear un **proyecto** en Supabase. Segundo, crear la **tabla** con sus columnas y el tipo de cada una: en el editor de tablas o con una sentencia SQL. Tercero, crear la **credencial** en n8n con la URL del proyecto y la clave de servicio (*service role*), que se copian desde la configuración de la API del proyecto.

```
create table pedidos (  
  id          bigint generated always as identity primary key,  
  creado_en   timestampz not null default now(),  
  email       text not null,  
  comuna      text not null,  
  cantidad    integer not null check (cantidad > 0),  
  precio_unitario integer not null,  
  total       integer not null  
);
```

EJEMPLO · MERCADO AUSTRAL · Qué hace cada línea de la tabla

`id` se numera solo y es la **clave primaria**: identifica cada pedido sin repetirse. `creado_en` toma la fecha y hora al guardar. `cantidad` debe ser un entero mayor que cero: la base de datos rechaza cualquier otro valor, aunque el workflow lo deje pasar. La tabla completa del curso agrega `nombre`, `tipo_cliente` y `producto`.

ERROR FRECUENTE · La clave a la vista

La clave de servicio salta las reglas de seguridad de la base de datos: quien la tiene puede leer y borrar todo. **Nunca** se pega en un parámetro de un nodo, en un mensaje ni en una captura para tu portafolio. Vive solo en la credencial de n8n. Si se expone, se genera una nueva en Supabase.

9 CRUD con el nodo Supabase

CONTENIDO DEL PLAN OPERACIONES CRUD BÁSICAS. · MANEJO DE DATOS RELACIONALES

CRUD reúne las cuatro operaciones básicas sobre datos: crear (*Create*), leer (*Read*), actualizar (*Update*) y borrar (*Delete*). El nodo Supabase las ofrece como *Create a row*, *Get a row* y *Get many rows*, *Update a row* y *Delete a row*. Para crear, puedes dejar que n8n asocie los campos del ítem con las columnas del mismo nombre (*Auto-map input data to columns*) o definir cada columna a mano.

Operación	Qué hace	Ejemplo en Mercado Austral
Create a row	Inserta una fila nueva	Guardar cada pedido que llega del formulario
Get a row / Get many rows	Lee una fila o varias, con filtros	Traer los pedidos de una comuna
Update a row	Cambia columnas de las filas que cumplen una condición	Marcar un pedido como despachado
Delete a row	Borra las filas que cumplen una condición	Eliminar los registros de prueba

Los **datos relacionales** son datos repartidos en tablas que se conectan por un campo. En vez de copiar todos los datos del pedido en cada devolución, la tabla `devoluciones` guarda solo el `id` del pedido al que pertenece, en una columna `pedido_id` que la base de datos declara como **clave foránea**. Así, un pedido puede tener varias devoluciones y ninguna puede apuntar a un pedido que no existe.

EJEMPLO · MERCADO AUSTRAL · Registrar una devolución

Llega una devolución del pedido 7. El workflow usa **Get a row** para traer el pedido con `id` igual a 7 y comprobar que existe. Después, **Create a row** en `devoluciones` con `pedido_id` = 7, el motivo y la fecha. Para un reporte que muestre juntos el pedido y su devolución, un Merge por el campo del `id` las une.

ERROR FRECUENTE · La fila rechazada

Si Supabase responde `invalid input syntax for type integer`, un campo numérico llegó como texto, por ejemplo `"3 "` con un espacio. La corrección está antes del nodo Supabase: en Edit Fields, `{{ Number(String($json.cantidad).trim()) }}` con el tipo Number.

En síntesis

- Los datos viajan como ítems JSON; mirar las vistas Table, JSON y Schema es la primera herramienta para entenderlos.
- Edit Fields crea y corrige campos, Filter descarta, Summarize agrupa y calcula, Split Out divide listas, Merge une flujos e If y Switch deciden el camino.
- Las expresiones van entre dobles llaves y se comprueban en la vista previa antes de ejecutar.
- Normaliza y declara los tipos apenas entran los datos: casi todos los errores de este aprendizaje nacen de un espacio de más o de un número guardado como texto.
- n8n trabaja en JSON; CSV y XML se leen y se generan con nodos específicos.
- Lo secreto va en credenciales, lo que cambia según el ambiente va en variables.
- Supabase guarda los datos en tablas relacionadas, y el nodo Supabase hace las cuatro operaciones CRUD.

SIGUIENTE PASO

Para practicar

- **Actividad 1, "Del formulario a la base de datos", partes B y C.** Construyes los workflows "Pedido a registro" y "Resumen semanal". Usan todo lo de esta lectura, y el resultado correcto del resumen es el del ejemplo de la sección 2.
- **Tutorial guiado "Tu primer workflow en n8n", pasos 6 a 9.** Normalización con Edit Fields y la primera fila en Supabase, paso a paso.
- **Video interactivo "Expresiones y depuración".** Predice el resultado de una expresión antes de verlo.
- **Documentación oficial.** La de n8n (docs.n8n.io) describe cada nodo con sus parámetros, y la de Supabase (supabase.com/docs), la creación de tablas y claves.

Autocomprobación

Responde con tus palabras antes de mirar las respuestas. Si dudas en alguna, vuelve a la sección que se indica.

- 1 Tu Summarize agrupa por comuna y el resultado muestra "Ñuñoa" y "Ñuñoa " como dos grupos distintos. ¿Qué pasó y en qué nodo lo corriges?
- 2 Quieres agregar a cada pedido la zona de despacho de su comuna, que está en otra tabla. ¿Qué modo del Merge usas y qué pasa si eliges todas las combinaciones posibles?
- 3 ¿Por qué la clave de servicio de Supabase va en una credencial de n8n y no escrita en un parámetro del nodo?

RESPUESTAS

1. Una de las comunas trae un espacio al final, y para el Summarize son textos distintos. Se corrige en el Edit Fields que va antes del Summarize, con `{{ $json.comuna.trim() }}`. Normalizar siempre va antes de agrupar (secciones 2 y 4).
2. Combine por Matching Fields, con el campo `comuna` en las dos entradas. Con todas las combinaciones, cada pedido se repite tantas veces como comunas tiene la tabla, y se detecta contando los ítems a la salida (sección 3).
3. La credencial se guarda cifrada y no aparece en el workflow exportado ni en las capturas. La clave de servicio salta las reglas de seguridad de la base de datos, así que quien la vea puede leer y borrar todos los datos (secciones 7 y 8).

Glosario

Clave foránea	Columna que guarda el identificador de una fila de otra tabla y así las relaciona, como <code>pedido_id</code> en <code>devoluciones</code> .
Credencial	Dato de acceso a otro servicio que n8n guarda cifrado, fuera del workflow.
CRUD	Las cuatro operaciones básicas sobre datos: crear, leer, actualizar y borrar.
Edit Fields (Set)	Nodo que crea, modifica o elimina campos de los ítems y declara su tipo.
Expresión	Código entre dobles llaves que n8n evalúa para cada ítem, como <code>{{ \$json.email }}</code> .
Ítem	Unidad de datos que pasa de un nodo a otro en n8n; es un objeto JSON con campos y valores.
Merge	Nodo que une los ítems de dos entradas, a continuación o combinándolos por un campo, por posición o en todas sus combinaciones.
Summarize	Nodo que agrupa los ítems por uno o más campos y calcula sumas, conteos o promedios por grupo.